



A simulated annealing-based recommender system for solving the tourist trip design problem

Takwa Tlili *, Saoussen Krichen

LARODEC Laboratory, Institut Supérieur de Gestion, Université de Tunis, Tunisia

ARTICLE INFO

Keywords:

Tourist trip design problem
Orienteering problem
Recommender system
Simulated annealing
k-means

ABSTRACT

Tourism recommender system (TRS) experienced high growth over the last decades given its significant role in tourist satisfaction. Tourists are usually worried about planning their trips since it requires a considerable effort and it is a time consuming task. The main purpose of TRS is to select points of interest (POIs) that match users preferences and to suggest personalized daily sightseeing. Designing such a suitable traveling tour is modeled as a Tourist Trip Design Problem (TTDP). Numerous variants of the problem are presented in the literature such as the Team Orienteering Problem (TOP) evoked in this paper. The aim is to maximize the total collected score of visited POIs subject to distance and time constraints. For solving the TTDP, we propose a metaheuristic based on two steps (i) grouping the POIs into clusters based on the start and end locations and (ii) solving the routing problem for each cluster to generate the optimal routes. The proposed approach is a combination between k-means and simulated annealing algorithms, namely, KSA. Besides, we develop StayPlan which is a TRS that integrates KSA algorithm to tackle the TTDP. StayPlan incorporates three main modules which are, user preferences, data retrieving & scoring and TTDP solving & optimization. Computational experiments compare the performance of our proposed algorithm with four state-of-the-art methods. The results show the efficiency and effectiveness of KSA algorithm in solving the benchmark suite from the literature. KSA algorithm reaches the best-known solutions for 100% of tested instances. Additionally, our method outperforms all the considered algorithms in terms of CPU time, it is quite fast with an average run time of 31.49 s. Ultimately, an illustrative example of the adapted scenario is applied to Tunis city as a real case.

1. Introduction

Personalized trip planning have become the first request of tourists all over the world. Tourists are usually confused about selecting the most interesting sites to visit during their stay to better profit from their time. Each traveler has a set of constraints (e.g limited budget, travel duration, ...) and preferences (e.g historical sites, museums, restaurants, entertainment centers, ...) to take into account when planning his trip. The most interesting sites for a tourist are called Points Of Interest (POIs). Planning a POIs route is still an effort and time consuming task for a tourist. Currently, several tourists plan their holidays based on the information collected from web sites, social medias, magazine articles and guidebooks. The incapacity to find the best personalized plan to visit multiple POIs, motivates the need for a Tourist Recommender system (TRS). Designing such suitable tourist tour is modeled as a Tourist Trip Design Problem (TTDP) that aims to optimize the travelers satisfaction. The objective function is about maximizing the overall profit collected by visiting POIs in all routes without violating a set of associated constraints. In this work, we introduce a personalized

TRS that attempts to derive multiple-day trips for tourists by solving the TTDP. The travelers can visit many attractions that maximize their satisfaction while respecting their constraints and preferences. Various extensions of TTDP have been considered in the literature, one of them is the Orienteering Problem (OP) that we evoke in the current work. To cope with OP, a hybrid evolutionary metaheuristic, named HEA is proposed. HEA can be defined as a combination of K-means method, partition crossover and hill climbing techniques. Once the HEA approach is developed, we integrate it into a TRS to offer an efficient visualized solution for OP. The numerical experiment compares the performance of our proposed algorithm with other state-of-the-art methods for solving OP benchmark instances. Finally, to test the system performance, we hold a real-case example using a real data set from Tunisia.

The remainder of this paper, is organized as follows. Section 2 is a survey of (1) TTDP extensions, OP solving methods and (2) TRS classes. Section 3 highlights the major contributions of our work. Section 4 describes the developed recommender system (StayPlan) and details the

* Corresponding author.

E-mail addresses: takwa.tlili@isamm.uma.tn (T. Tlili), saoussen.krichen@isg.rnu.tn (S. Krichen).

different modules integrated in the architecture. Section 5 presents the proposed solution approach named KSA, by reporting its main methods and pseudo-codes. Computational results and KSA performance evaluation are reported in Section 6. In Section 7, a real-case example is presented in order to test the system performance. Conclusions and future work presented in Section 8.

2. Position in literature and related work

This section provides a brief review of different topics directly related to our work, including: tourist trip design problem, orienteering problem and the methods proposed to solve it, and tourist recommender systems.

2.1. Tourist trip design problem

The selection and scheduling of POIs is not a simple task that can be done rapidly without losing a time. This task has to be generated automatically by a robust software integrating routing algorithms. This issue is referred to in the literature as the Tourist Trip Design Problem (TTDP) (Vansteenkeweg & Van Oudheusden, 2007) which is essentially about organizing daily sightseeing that matches the tourist preferences. The key objective is to maximize the total collected profit from POIs while respecting a set of additional constraints such as: required time to visit the sites, opening days/hours, entrance fees, weather conditions, ... (Dibbelt et al., 2017). The TTDP is considered as a variant of the Orienteering Problem (OP) (Expósito, Mancini, Brito, & Moreno, 2019) or an extension of the Traveling Salesman Problem with Profits (TSPP) (Vansteenkeweg & Gunawan, 2019). A good number of works have studied the TTDP and a set of constraints have been included to define numerous variants that can be classified into two groups (Gavalas, Konstantopoulos, Mastakas and Pantziou, 2014). (1) Multiple tour TTDP and (2) Single tour TTDP. Multiple tour TTDP can be designed as a Vehicle Routing Problem (VRP) with profits where the profit is associated with each POI gained from multiple vehicles. The objective is about maximizing the difference between the total score and the total travel cost. Another extension of multiple tour category is the prize-collecting VRP (Trachanatzi, Rigakis, Marinaki, & Marinakis, 2020) which aims to optimize both of travel cost and number of vehicles used. Single tour TTDP aims to find only one path between POIs that maximizes the profit and minimizes the travel cost. This problem can be simulated to the TSPP (Gunawan, Lau, Vansteenkeweg, & Lu, 2017). Although the most well known problem in this category is the Orienteering Problem (OP) in which the POIs should be visited within a time windows (Jriji, Krichen, & Madany, 2020). In this paper, the TTDP is designed as the aforementioned OP that will be reviewed in the next section.

2.2. Orienteering problem

The Orienteering Problem (OP) was introduced by Golden, Levy, and Vohra (1987). The primary goal of OP is to find the path that maximizes the total profit without violating distance or time constraint. Recently, many studies have tackled different variants of the problem. According to Gavalas et al. (2014), OP with time windows is an extension of the problem in which every node should be visited during a predefined time. The time-dependent OP is another variant where the travel time depends on the hour of the day. It can be more effective while solving multi-modal transports between POIs (Verbeeck, Vansteenkeweg, & Aghezzaf, 2016). The generalized OP was handled in the work of Pěnička, Faigl, Váňa, and Saska (2017) and Urrutia-Zambrana, Tirado, and Mateos (2021), it is described as the problem where each of the vertices is correlated to more than two scores. The objective function is now about finding the best itinerary by calculating a weighted sum of all the collected scores. Bianchessi, Mansini, and Grazia Speranza (2018) evoked the stochastic OP which is very interesting in the case of congestion, it can be seen as a

time-dependent OP with random values for some factors. Team OP (TOP) is the most well known variant of OP (Hu, Fathi, & Pardalos, 2018a) since it has a lot of real world applications especially in the tourism field. It is about both selecting the set of nodes to visit and finding the shortest itinerary to do it. For more details about TOP variants, the interested readers are referred to the paper of Gunawan et al. (2017). OP has been studied for many years (Gunawan et al., 2017; Vansteenkeweg, Souffriau, Berghe, & Van Oudheusden, 2011). So far, researchers have developed different methods to cope with the problem. Due to the NP-hardness of OP, an extensive number of metaheuristics have been developed to solve the problem. In the paper by Santini (2019), the author proposed a new heuristic algorithm which is based on the adaptive large neighborhood search paradigm derived from the clustering of the graph. In Tsakirakis, Marinaki, Marinakis, and Matsatsinis (2019), the authors introduced a novel metaheuristic named similarity hybrid harmony search that imitates the natural music improvisation process. The musicians try to find the right acoustic frequency similar to the way the algorithm seeks the best solution while respecting the constraints. More recently, a learn heuristic approach, which is a combination of heuristics with machine learning, has also been investigated in the work of Bayliss, Juan, Currie, and Panadero (2020). This approach integrated the instance-based learning algorithm with interpolated predictions to handle the aerial drone OP. Besides, nature inspired algorithms have received a great deal of attention for solving the OP. We cite, artificial bee colony (Martin-Moreno & Vega-Rodriguez, 2018), simulated Annealing (Lin & Vincent, 2015a; Zhou, Li, & Li, 2018), and particle swarm optimization (Vincent, Perwira Redi, Jewpanya, & Gunawan, 2019). Evolutionary algorithms for solving the OP are described in the following works (Aghdasi, Saeedvand, & Baltes, 2019; Bossek, Grimme, Meisel, Rudolph, & Trautmann, 2019; Hu, Fathi, & Pardalos, 2018b; Karabulut & Fatih Tasgetiren, 2020).

2.3. Tourist recommender system

Tourist Recommender system (TRS) can be viewed as a personalized tool that provides the tourists with the suggestions best matching their choices. They analyze the behavior of interested users in order to collect their preferences. The key objective of TRS is to propose suggestions to tourists and support them in organizing their trips. Three features are considered to categorize TRSs: (1) Recommendation approaches, (2) Human interface style, and (3) Artificial intelligence techniques.

For TRSs integrating optimization framework, depends mostly on recommendation approaches. TRSs which are based on the filtering algorithms implemented to recommend POIs to tourists. The best known classes in this category are: content-based, collaborative and demographic systems (Montaner, López, & De La Rosa, 2003). Content-based systems works with the items rated by users in their profiles to make the recommendation. The content is then analyzed to identify the users interest. Otium, proposed by Montejo-Ráez, Perea-Ortega, García-Cumbreras, and Martínez-Santiago (2011), is a web-based TRS that plans for leisure and tourism. This system helps Spanish users in the creation of the visit schedule generated according to their profiles. In collaborative systems, the recommendations usually depends on the behaviors of the users. Therefore, the users with common preferences are grouped by comparing their ratings. The collaborative approach can be divided into two methods the memory-based and model-based. The former operates over the entire user database to make predictions. An example of these systems is GuideMe (Umanets, Ferreira, & Leite, 2014) which is a tourist guide system that makes the recommendation according to the best rated POIs. Demographic systems in which the recommendation process is based on user demographic characteristics like age, civil status and language, where users are matched by their common demographic profile. This approach is used by SigTur/E-Destination proposed by Moreno, Valls, Isern, Marin, and Borràs (2013) to give a personalized recommendation for the tourists of Tarragona. The combination of the above mentioned

systems forms the hybrid recommender systems. These tools aim to avoid the shortcomings of the other systems and reduce their deficiencies. For example, in [Schiaffino and Amandi \(2009\)](#), the authors developed an expert agent system called Traveler, which combines collaborative with Content-based and demographic recommendation systems. It designed to suggest package holidays and tours to the users that are more interesting, according to their profile. Another class of recommender systems integrates session-based recommendation algorithms that typically aim to predict the user's immediate next actions ([Ludewig & Jannach, 2018](#); [Quadrana, Cremonesi, & Jannach, 2018](#)). More recently, [Esmaeili, Mardani, Hashemi Golpayegani, and Madar \(2020\)](#) presented a personalized list of POIs according to the similarity of tourist interests, trust, reputation, relationships, and social communities. [Logesh, Subramaniaswamy, and Vijayakumar \(2018\)](#) developed a personalized context-aware hybrid TRS integrating user's contextual information.

3. Contributions

In this paper, we make the following contributions:

- We model the issue as a well-known optimization problem named TTDP. In order to solve the problem, we propose a hybrid metaheuristic running in two steps:
 - (1) Grouping the POIs into clusters based on the start and end locations. The clustering stage is handled by K-means algorithm.
 - (2) Solving the routing problem for each cluster to generate the optimal routes. For this subproblem we develop a simulated annealing method.
 The proposed approach is a combination between k-means and simulated annealing algorithms, namely, KSA.
- We design a TRS, named StayPlan, that supports the tourist with oriented recommendations to maximize the entertainment value of visiting the POIs according to their preferences. StayPlan is recommender system that integrates KSA algorithm to tackle the TTDP. The proposed architecture incorporates three main modules which are, (1) user preferences, (2) data retrieving & scoring and (3) TTDP solving & optimization. To accomplish the implementation of the proposed TRS, we used TripAdvisor framework and Google Maps to extract the POIs scores and the geographical localization, respectively.
- We demonstrate the validity of our approach by performing numerical experiments. Therefore, we carry out a comparative study with recent state-of-the-art algorithms using benchmark instances. The results show the outperforming and effectiveness of KSA algorithm in solving the TTDP, more specifically the team orienteering problem.

4. StayPlan recommender system design

The proposed system assists the users to plan personalized daily-tours for their trip. For this purpose, three key stages are to be followed:

- Collect information about user's centers of interest, budget and time constraints.
- Select potential places to visit and identify their scores using TripAdvisor.
- Propose a cluster first-route second algorithm to generate optimal tours of a trip.

Scenario steps description

1. The tourist enters demographic data via the user interface, e.g user's name, age, provenance, ...
2. The tourist provides his preferences about centers of interest such as visiting museums, theaters, monuments, restaurants, ...

3. The tourist specifies his actual location (hotel, accommodation), the final destination, the stay duration and the travel way (car, public transport, walking, ...)
4. From the POIs database, the system selects a refined list of places according to the categories selected by the user.
5. TripAdvisor provides the ratings of the selected destinations. Thereafter, the scoring is calculated in order to generate the most suitable list for the user.
6. Geolocalization of POIs is a key step before the clustering process. Hence, Google Maps service is used to obtain places coordinates and real distances between all points.
7. Optimization component integrates the implemented algorithm that begins by grouping the POIs into clusters according to the provided information. Then, it solves the routing problem for each cluster to offer a trip planning with daily routes.
8. The suggested routes are displayed by the viewer service in a map showing the suggested tours.

The detailed architecture framework is presented in [Fig. 1](#) that describes the designed modules, (i) user information (Section 4.1), (ii) retrieving data and scoring (Section 4.2) and (iii) optimize and generate tours (Section 4.3).

4.1. User preferences module

StayPlan is a friendly-user system that easily allows the tourist to define his information and preferences. The tourist inputs demographic data via the user interface, such as the name, age, provenance, etc. Besides, he chooses his preferences and centers of interest categories from a predefined list. Moreover, the tourist should define the start and end location for each day (hotel, accommodation, etc.), the stay duration and the travel way (e.g car, public transport, walking).

4.2. Data retrieving and scoring module

StayPlan collects the most interesting POIs, according to user preferences, from TripAdvisor platform (www.tripadvisor.com). TripAdvisor is a tourist social network with a virtual community that maintains a data base created by crowdsourcing where the minimal provided data is the POI. Tourists post their opinions and recommendations based on their trip experiences. A variety of places categories are available in TripAdvisor such as Theater, Art and Performance, Parks, Gardens and Nature and Museums, Zoos and Aquariums. StayPlan offers six main alternatives of category to select by the tourist: (1) Sights and Museums, (2) Outdoors and recreation, (3) Restaurants, (4) Shopping, (5) Theaters and (6) Sport events. We rely on the total TripAdvisor rating and the number of users votes to determine the score of each place. Simultaneously, StayPlan creates a personalized POIs list and computes the scores that reflect the relevance of POIs. The final score of a place is calculated as follows [Wörndl, Hefe, and Herzog \(2017\)](#).

$$score = rating \times \log_2 votesnumber + 1$$

At the end of this module, a personalized POIs list sorted by scores is generated to build an optimized trip planning.

4.3. Solving the TTDP and optimization module

The main contribution of the present paper is the development of an efficient optimization service based on metaheuristics. Once all the places in the relevant area have been categorized and scored, two steps are considered for the optimization service.

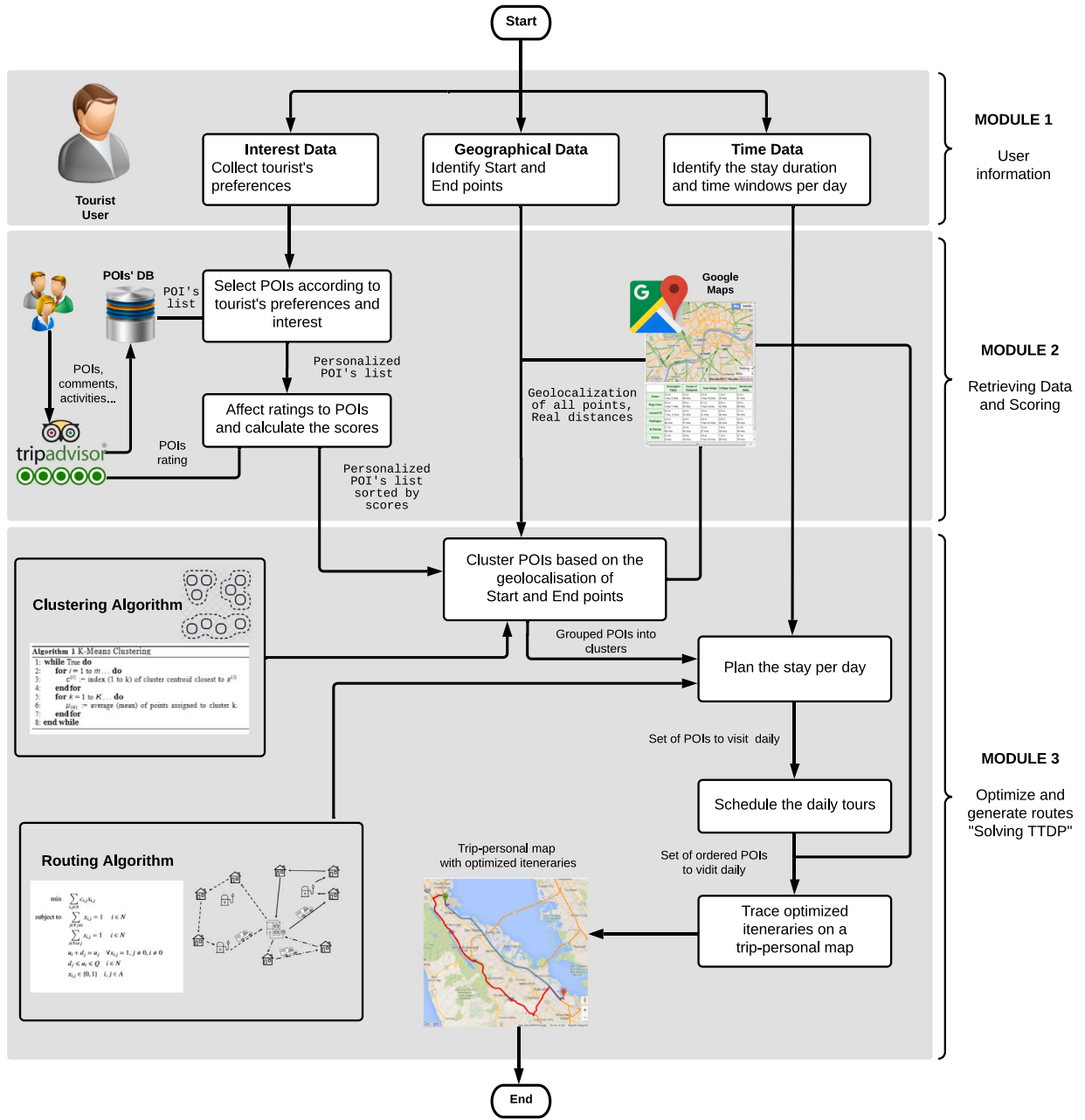


Fig. 1. StayPlan architecture.

Step 1: Clustering This step is about grouping the POIs into clusters based on the start and end locations. We use the k-means algorithm (Section 5.2), where the input is the geographical coordinates of all places and the number of clusters is equal to the number of available days. Localizations of the start and end points as well as the destinations are extracted from Google Maps (<http://www.google.com/maps/>).

Google Maps is a web-based service that provides detailed information about geographical regions and sites around the world. It offers several services like, (i) Google Street View that offers the option to view and navigate through horizontal and vertical panoramic street, (ii) Google Places API that allows searching for place information using numerous categories.

Step 2: Routing This step operates into the generated clusters, i.e. For each day, StayPlan proposes the scheduling and the optimal itinerary to visit the places. Since the problem is modeled as

a TTDP, the main objective is maximizing the total scores of visited POIs.

In terms of combinatorial optimization field, the problem is formulated as a Mixed-Integer Programming (MIP) as follows.

$$\text{Max} \sum_{i \in P} \sum_{j \in D} \text{Score}_i Y_i^j \quad (1)$$

where, P is the number of POIs, Score_i is the score of POI i and D is the number of available days. Y_i^j is a binary variable; $Y_i^j = 1$ if POI i is visited in day j ; $Y_i^j = 0$ otherwise, $\forall i \in P$ and $\forall j \in D$.

The TTDP objective function (1) seeks to maximize the total collected score from POIs which is the tourists ratings retrieved in TripAdvisor.

A set of requirements should be respected in order to obtain feasible and realistic solutions. The constraints in the evoked problem are listed as follows.

- Each daily tour starts from a depart location and arrives at a ends location prefixed by the user.
- Each POI should be visited only once during the entire stay.
- A POI can be visited by a given route only if this POI has been assigned to the corresponding day.
- The maximum route duration is to be respected in all daily routes.

In StayPlan, the integrated optimizer seeks to solve the problem in two steps: (1) Clustering step using K-means algorithm and (2) Routing step using a swarm-based metaheuristic. Finally, the generated itineraries are visualized in a personalized map for each day. Section 5 explains the developed approach in details.

5. KSA resolution approach

For the TTDP in which the POIs naturally form partitions, the performance can be improved by first clustering the POIs, and then applying the SA iteratively to each cluster to generate the optimal routing. In Fig. 2, (1) the start and end points are overlaid, i.e. the tourist leaves and returns to the same place (hotel) and (2) given a set of POIs to be distributed on days then scheduled to generate the daily itinerary.

The optimization module in StayPlan is composed of two parts which are the clustering and the routing, detailed in Sections 5.1 and 5.2.

5.1. Clustering algorithm

The K-means algorithm (MacQueen et al., 1967) is one of the most well-performed clustering algorithms widely applied to solve routing problems. The main concept of K-means is to obtain k centroids, and each centroid corresponds to a cluster. Initially, the developed method starts with a set of initial k cluster centers, where k is equal to the number of available days (prefixed by the user). Then it uses an iterative process to improve the cluster quality. In each iteration, (1) K-means calculates the Euclidean distance between the set of POIs and centroids, (2) each POI is assigned to the nearest cluster. Finally, k centroids with the corresponding POIs are obtained.

Then all the centroids are updated to the average of each cluster according to Eq. (2).

$$C_j = \frac{1}{v_j} \sum_{i=1}^{v_j} x_{ij} \quad (2)$$

where:

- C_j : Clustering center j
- O_j : Number of POIs of cluster j
- x_{ji} : POI i of cluster j

The main steps of the developed k-means are summarized as follows:

1. Determine first set center.
2. Calculate the Euclidean distance between specified POI and center points. All objects are assigned to the nearest cluster.
3. Update clustering centers.
4. Repeat step 2 and 3 until center points are not changed.

In StayPlan, we use the geographical coordinates of the POIs, extracted from Google Maps. Then, iteratively we allocate POIs to the cluster whose centroid is closest to that place.

5.2. Simulated annealing

Simulated annealing (SA) is a local search metaheuristic proposed first by Metropolis in Metropolis, Rosenbluth, Rosenbluth, Teller, and Teller (1953). It is a probabilistic technique based on the Monte-Carlo iterative solution scheme. SA algorithm draws its concept from the annealing procedure of physical solid matters. The annealing process starts from a high initial temperature, then the metal is heated and slowly cooled to a lower temperature. Analogically, SA adopts an iterative movement according to the temperature deviation to mimic the metallurgical transactions. This method is capable of escaping from local optimum by accepting, worse solutions with a probability according to the state of the system. Moreover, SA method needs more diversification mechanisms to avoid being confined in a small region of the solution space and trapped at local optima (Lin & Vincent, 2015b).

5.2.1. General outline

This analogy can be used in TTDP, in which the state of solid represents the feasible solution, the energy at each state represents the improvement in the objective function and the maximum energy state is the optimal solution.

At each iteration of the SA, a new neighbor x^* is taken from the current solution neighborhood $N(x)$. If the move to the neighbor increases the objective function value $f(x)$ or leaves it unchanged, then the move is always accepted. Thus, the solution x^* becomes the new current solution if $\delta T_0 < 0$, where $\delta = f(x^*) - f(x)$ and T_0 is the initial temperature. At the end of an iteration, gradual attenuation of T is expressed as αT_0 , where α is the temperature cooling parameter.

Algorithm 1 SA algorithm for TTDP

Require: T_0 , α , x and $PopSize$

Ensure: x^* : The best solution

```

1: while time limit is not exceeded do
2:   for i=1 to PopSize do
3:     Generate  $r \sim U(0, 1)$ 
4:     if  $r \leq 1/3$  then
5:       Generate new solution  $x^*$  using point relocation operator
6:     else if  $r > \frac{1}{3}$  &&  $r \leq \frac{2}{3}$  then
7:       Generate new solution  $x^*$  using point exchange operator
8:     else
9:       Generate new solution  $x^*$  using route interchange operator
10:    end if
11:    Calculate  $\delta T$ 
12:    if  $\delta T < 0$  then
13:      Accept  $x^*$  as the new solution
14:    else
15:      Generate  $p \sim U(0, 1)$ 
16:    end if
17:    if  $p < e^{\frac{\delta T}{T}}$  then
18:      Accept  $x^*$  as the new solution
19:    end if
20:  end for
21:   $T = \alpha T_0$ 
22: end while
23: return  $x^*$ 

```

5.2.2. Neighborhood structures

When the proposed SA attempts to move to a new solution, one of three employed moves is selected. Let $N(x)$ denotes the set of neighboring solutions of the current solution x . At each iteration, a new feasible solution x^* is selected from $N(x)$. The first type is *point relocation* in which a POI in a route is deleted and relocated to another random position in a different route. The second move is the *point exchange* that swaps positions of two POIs in two routes. The third is the *route interchange* in which a couple of POIs is randomly selected and the segment of the first route before the selected POI is connected with

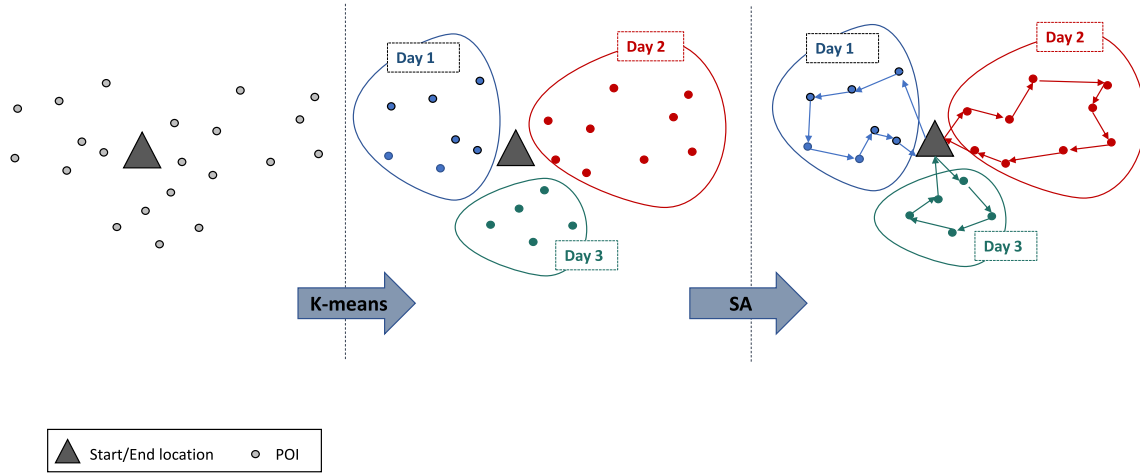


Fig. 2. Resolution method steps for an example of 21 POIs and 3 available days.

the second segment of the other route after the other selected POI (see Fig. 3).

6. Computational experiments

Our approach described in Section 5 is coded in Java. The testing environment is a computer equipped with an Intel Core i7-7500U processor running at 2.9 GHz, 8 GB of RAM. In this section, we report and analyze the experimental results carried on to validate the proposed KSA which is integrated in StayPlan. Ten independent runs are performed for each benchmark instance and the best obtained solutions are reported.

6.1. Dataset description and parameters setting

In order to compare the performance of KSA with those of the state-of-the-art algorithms, we selected and tested 156 instances from the well-known benchmark of team orienteering problem by Chao, Golden, and Wasil (1996) that can be downloaded from [http://www.mech.kuleuven.be/en/cib/op.Chaoetal.\(1996b\)](http://www.mech.kuleuven.be/en/cib/op.Chaoetal.(1996b)). The standard data set is divided into seven sets based on the number of control points ranging from 21 to 102.

In this work, we tested four sets (set4, set 5, set 6 and set 7). In each set, the number of vertices and their coordinates are identical. However, the instances differ for the number of vehicles that varies between 2 and 4 as well as the value of the maximum tour duration.

In order to calibrate the parameters of our algorithm, we carried out preliminary experiments on random instances from the benchmark sets described beforehand. We started from the most used settings taken from the previous studies to solve TOP problems, then tuned each parameter by varying the base value. For each parameter, we retain the best configuration found before proceeding to tune the next one. KSA needs four parameters to run. The initial temperature T_0 has a considerable impact on the solution quality. For this reason, an extensive calibration on the selected instances was carried out to determine T_0 . As SA converges slowly on such NP-hard optimization problem, a low initial temperature T_0 is preferred and is set to 1.0. Through preliminary tests on the data set instances, the cooling rate $\alpha = 0.9$ significantly leads to the best results. This value is suggested in the literature (Chen & Shi, 2019) and (Kim, Kim, & Noh, 2020). For each instance, KSA is run for 10 times with random seeds 1–10 and the population size is set to 30 solution.

6.2. Comparison with recent existing algorithms

In order to check the performance of the developed KSA algorithm, it is necessary to compare it with other algorithms from the literature for benchmarking purposes. We perform the comparison with the last four state-of-the-art algorithms from the literature. For the sake of completeness, we report the different computing environments used to test the four solution methods.

- HSSH: Similarity Hybrid Harmony Search (static version) by Tsakirakis et al. (2019), tested on Intel Core i7-2670QM, 2.20 GHz.
- HSSH2: Similarity Hybrid Harmony Search (dynamic version) by Tsakirakis et al. (2019), tested on Intel Core i7-2670QM, 2.20 GHz.
- LNS: Large Neighborhood Search by Orlis, Bianchessi, Roberti, and Dullaert (2020), tested on Intel Core i7-6700U, 4.00 GHz.
- HALNS: Hybrid Adaptive Large Neighborhood Search by Hammami, Rekik, and Coelho (2020), tested on Intel Core i7-7700HQ, 2.80 GHz.

Tables 1–4 summarize the test results of our proposed approach and those of the state-of-the-art algorithms for all benchmark instances. The first two columns describe the instances and report, the name and the best known solution (BKS), respectively. For each algorithm, columns ‘Best’ report the best found solutions, columns ‘Gap’ report the average percentage deviation between the KSA objective values and BKS (3) and columns ‘CPU’ report the average execution time in seconds. In the last row of the tables, we note the total average of each column.

The bold values means the best-so-far results found by the algorithms and — means not available value.

$$Gap(\%) = \frac{BKS - Best}{BKS} \times 100 \quad (3)$$

On the basis of Table 1, we conclude that our proposed algorithm solves all the instances of Set 4 to optimality, which is the case of LNS and HALNS. For HSSH, the average Gap is 1.66% and the best known solution is found in only 16.9% of instances. Those values are improved in the second version HSSH2 when (1) the Gap dropped to 0.27%, and (2) the percentage of instances solved to optimality rises to 47.17%.

Regarding the computational time, KSA is quite fast compared to the other algorithms with average CPU time of 31.49 s against 107.3, 109.6, 218 and 32.24 s for HSSH, HSSH2, LNS and HALNS, respectively.

Now, we analyze the performance of algorithms on instances of Set 5. From Table 2, it can be seen that the proposed KSA algorithm and HALNS are so competitive in terms of Gap (0%) but our approach performs faster with computational time equal to 10.1 s. However, for

Table 1

Detailed computational results of KSA on benchmark instances of set 4.

Inst		SHHS			SHHS2			LNS			HALNS			KSA		
Name	BKS	Best	Gap	CPU	Best	Gap	CPU	Best	Gap	CPU	Best	Gap	CPU	Best	Gap	CPU
p4.2.a	206	206	0.0	4.7	206	0.0	4.3	206	0.0	5.6	206	0.0	–	206	0.0	1.3
p4.2.b	341	341	0.0	14.5	341	0.0	8.1	341	0.0	25.3	341	0.0	–	341	0.0	2.4
p4.2.c	452	452	0.0	14.8	452	0.0	13.1	452	0.0	56.1	452	0.0	–	452	0.0	2.8
p4.2.d	531	528	0.56	16.7	531	0.0	20.5	531	0.0	93.2	531	0.0	–	531	0.0	5
p4.2.e	618	611	1.13	33.6	613	0.81	41.6	618	0.0	130.0	618	0.0	–	618	0.0	4.7
p4.2.f	687	673	2.04	38.5	684	0.44	43	687	0.0	150.3	687	0.0	–	687	0.0	6.6
p4.2.g	757	753	0.53	65	757	0.0	72.8	757	0.0	159.5	757	0.0	–	757	0.0	18.6
p4.2.h	835	816	2.28	70.9	827	0.96	75.4	835	0.0	175.3	835	0.0	–	835	0.0	41
p4.2.i	918	896	2.40	71.8	912	0.65	76.4	918	0.0	191.7	918	0.0	–	918	0.0	41.9
p4.2.j	965	936	3.01	75	957	0.83	73.3	965	0.0	218.5	965	0.0	–	965	0.0	45.1
p4.2.k	1022	983	3.82	96	1010	1.17	103.1	1022	0.0	237.2	1022	0.0	–	1022	0.0	66.1
p4.2.l	1074	1048	2.42	99.8	1074	0.0	113.5	1074	0.0	263.9	1074	0.0	–	1074	0.0	39.3
p4.2.m	1132	1078	4.77	97	1125	0.62	98.9	1132	0.0	286.8	1132	0.0	–	1132	0.0	35.2
p4.2.n	1174	1152	1.87	101.4	1168	0.51	106.1	1174	0.0	328.2	1174	0.0	–	1174	0.0	49.6
p4.2.o	1218	1171	3.86	126.6	1216	0.16	119.8	1218	0.0	347.9	1218	0.0	–	1218	0.0	42.8
p4.2.p	1242	1211	2.50	135.2	1242	0.0	137.3	1242	0.0	410.3	1242	0.0	–	1242	0.0	50.5
p4.2.q	1268	1245	1.81	143	1259	0.71	146.7	1268	0.0	454.3	1268	0.0	–	1268	0.0	12.7
p4.2.r	1292	1269	1.78	135.4	1282	0.77	140.7	1292	0.0	485.0	1292	0.0	–	1292	0.0	30.1
p4.2.s	1304	1284	1.53	158.3	1294	0.77	165.8	1304	0.0	535.8	1304	0.0	–	1304	0.0	31.8
p4.2.t	1306	1302	0.31	169.4	1306	0.0	171.2	1306	0.0	19.7	1306	0.0	–	1306	0.0	43.7
p4.3.d	335	335	0.0	12.4	335	0.0	15.2	335	0.0	18.1	335	0.0	–	335	0.0	2.7
p4.3.e	468	468	0.0	14.1	468	0.0	18.8	468	0.0	35.4	468	0.0	–	468	0.0	15.4
p4.3.f	579	579	0.0	19	579	0.0	23.9	579	0.0	70.5	579	0.0	–	579	0.0	2
p4.3.g	653	651	0.31	44.8	653	0.0	46.2	653	0.0	96.9	653	0.0	–	653	0.0	14.9
p4.3.h	729	715	1.92	47.7	729	0.0	49.3	729	0.0	150.0	729	0.0	–	729	0.0	17.8
p4.3.i	809	796	1.61	53.4	809	0.0	56.6	809	0.0	191.5	809	0.0	–	809	0.0	23.5
p4.3.j	861	854	0.81	59.9	858	0.35	63	861	0.0	202.2	861	0.0	–	861	0.0	8.1
p4.3.k	919	908	1.20	65.5	919	0.0	79.3	919	0.0	231.3	919	0.0	–	919	0.0	35.6
p4.3.l	979	954	2.55	96.3	973	0.61	107.1	979	0.0	239.7	979	0.0	–	979	0.0	31.1
p4.3.m	1063	1028	3.29	156.6	1063	0.0	147.2	1063	0.0	255.8	1063	0.0	–	1063	0.0	47.2
p4.3.n	1121	1093	2.50	167.4	1116	0.45	155.1	1121	0.0	270.0	1121	0.0	–	1121	0.0	44.6
p4.3.o	1172	1149	1.96	163.2	1168	0.34	158	1172	0.0	314.5	1172	0.0	–	1172	0.0	46.7
p4.3.p	1222	1213	0.74	182.7	1222	0.0	163.2	1222	0.0	335.6	1222	0.0	–	1222	0.0	36.7
p4.3.q	1253	1226	2.15	179.6	1251	0.16	177.1	1253	0.0	359.5	1253	0.0	–	1253	0.0	63.5
p4.3.r	1273	1247	2.04	196	1269	0.31	192.3	1273	0.0	368.0	1273	0.0	–	1273	0.0	63.1
p4.3.s	1295	1265	2.32	193.7	1285	0.77	189.4	1295	0.0	409.6	1295	0.0	–	1295	0.0	19.3
p4.3.t	1305	1278	2.07	175.2	1302	0.23	193.1	1305	0.0	422.8	1305	0.0	–	1305	0.0	41.7
p4.4.e	183	183	0.0	4.2	183	0.0	3.1	183	0.0	0.0	183	0.0	–	183	0.0	2.8
p4.4.f	324	324	0.0	14.6	324	0.0	11.6	324	0.0	9.4	324	0.0	–	324	0.0	4.6
p4.4.g	461	461	0.0	22.5	461	0.0	28.6	461	0.0	29.1	461	0.0	–	461	0.0	24.6
p4.4.h	571	556	2.63	46.4	571	0.0	53	571	0.0	42.4	571	0.0	–	571	0.0	16.5
p4.4.i	657	653	0.61	58	657	0.0	56.5	657	0.0	81.1	657	0.0	–	657	0.0	28.1
p4.4.j	732	731	0.14	55.2	732	0.0	59.1	732	0.0	112.8	732	0.0	–	732	0.0	25.3
p4.4.k	821	819	0.24	64.4	820	0.12	66.8	821	0.0	158.0	821	0.0	–	821	0.0	34.5
p4.4.l	880	859	2.39	145.4	878	0.23	156.6	880	0.0	231.3	880	0.0	–	880	0.0	32
p4.4.m	919	896	2.50	158.2	916	0.33	167.8	919	0.0	260.4	919	0.0	–	919	0.0	38.3
p4.4.n	976	948	2.87	159	971	0.51	162.8	976	0.0	262.8	976	0.0	–	976	0.0	42.2
p4.4.o	1061	1035	2.45	169.1	1061	0.0	166.3	1061	0.0	301.8	1061	0.0	–	1061	0.0	30.3
p4.4.p	1124	1112	1.07	258.2	1120	0.36	255.6	1124	0.0	321.9	1124	0.0	–	1124	0.0	52.3
p4.4.q	1161	1129	2.76	252.7	1158	0.26	267.3	1161	0.0	334.5	1161	0.0	–	1161	0.0	61.2
p4.4.r	1216	1170	3.78	246.5	1216	0.0	237.2	1216	0.0	342.1	1216	0.0	–	1216	0.0	85.1
p4.4.s	1260	1223	2.94	251	1252	0.63	258.7	1260	0.0	361.5	1260	0.0	–	1260	0.0	52.3
p4.4.t	1285	1264	1.63	284.6	1281	0.31	293.1	1285	0.0	375.9	1285	0.0	–	1285	0.0	53.9
AVG			1.66	107.3		0.27	109.6		0.0	218		0.0	32.24		0.0	31.49

the rest of algorithms, the best result of Gap is generated by HSSH2 with 0.004% that outperforms LNS (0.07%) and HSSH (0.23%).

The numerical results of the algorithm on set 6 can be observed in Table 3. Our method as well as the other algorithms find the BKS for the 15 tested instances, but KSA solves the problems during a minimal CPU time (8.9 s).

According to Table 4, the top algorithms with 0% of Gap, are LNS, HALNS and KSA. For HSSH and HSSH2, the Gap values are 1.52% and 0.27%, respectively. On the basis on the computational speed performance, the five algorithms are ordered as follows: KSA (28.46 s), HALNS (30.89 s), LNS (152.95 s), HSSH (86.91 s) and HSSH2 (86.91 s).

In Table 5, we provide:

- #Best: The number of instances on which the best-known solution was found.

- ARPD (%): The average relative percentage deviation over the 156 tested instances (sets 4–7). The calculation of ARPD is performed using the following equation: $ARPD = \sum_{i=1}^{156} \frac{(BKS_i - S_i^{KSA})}{156} \times 100$. S_i^{KSA} denotes the score of the solution found by KSA and BKS_i is the score of the best known solution of instance i .
- AVG CPU (sec): The average execution time.

Table 5 prompts a few observations:

- In terms of total number of best solutions, KSA and HALNS have the best performance by solving all the tested 156 instances to optimality. LNS failed to solve only a single instance which is 'p5.2.w' with a Gap of 0.32%. HSSH and HSSH2 attain the BKS in 42.95% and 66.67% of instances, respectively.
- It can be observed that for KSA and HALNS, the average relative percentage deviation is null since they found all the best

Table 2

Detailed computational results of KSA on benchmark instances of set 5.

Inst		SHHS			SHHS2			LNS			HALNS			KSA		
Name	BKS	Best	Gap	CPU	Best	Gap	CPU	Best	Gap	CPU	Best	Gap	CPU	Best	Gap	CPU
p5.2.h	410	410	0.0	16.4	410	0.0	14.2	410	0.0	38.1	410	0.0	–	410	0.0	4.5
p5.2.j	580	580	0.0	31.4	580	0.0	23.7	580	0.0	38.8	580	0.0	–	580	0.0	13.7
p5.2.k	670	670	0.0	29.5	670	0.0	19.6	670	0.0	43.2	670	0.0	–	670	0.0	4.2
p5.2.l	800	800	0.0	29.1	800	0.0	21.5	800	0.0	46	800	0.0	–	800	0.0	2.5
p5.2.m	860	860	0.0	29.6	860	0.0	17.8	860	0.0	49.2	860	0.0	–	860	0.0	2.4
p5.2.n	925	925	0.0	28.7	925	0.0	22.6	925	0.0	50.9	925	0.0	–	925	0.0	2
p5.2.o	1020	1020	0.0	29.6	1020	0.0	22.2	1020	0.0	54.8	1020	0.0	–	1020	0.0	2.9
p5.2.p	1150	1150	0.0	25.6	1150	0.0	19.8	1150	0.0	59.9	1150	0.0	–	1150	0.0	3.1
p5.2.q	1195	1195	0.0	29.6	1195	0.0	22.7	1195	0.0	64	1195	0.0	–	1195	0.0	2.9
p5.2.r	1260	1260	0.0	30	1260	0.0	27.8	1260	0.0	66.6	1260	0.0	–	1260	0.0	3.3
p5.2.s	1340	1325	1.12	40.2	1340	0.0	41	1340	0.0	71.7	1340	0.0	–	1340	0.0	13.5
p5.2.t	1400	1380	1.43	34.9	1400	0.0	38.4	1400	0.0	73.9	1400	0.0	–	1400	0.0	8.2
p5.2.u	1460	1460	0.0	44.5	1460	0.0	41.8	1460	0.0	77.8	1460	0.0	–	1460	0.0	17.8
p5.2.v	1505	1505	0.0	34.7	1505	0.0	29.9	1505	0.0	82.1	1505	0.0	–	1505	0.0	8
p5.2.w	1565	1560	0.32	36.9	1560	0.32	33.8	1560	0.32	85.8	1565	0.0	–	1565	0.0	10.2
p5.2.x	1610	1610	0.0	39.6	1610	0.0	46	1610	0.0	88.9	1610	0.0	–	1610	0.0	3.9
p5.2.y	1645	1645	0.0	38.5	1645	0.0	36.8	1645	0.0	91.3	1645	0.0	–	1645	0.0	11.8
p5.2.z	1680	1680	0.0	35.7	1680	0.0	36.5	1680	0.0	84.5	1680	0.0	–	1680	0.0	9
p5.3.k	495	495	0.0	22.5	495	0.0	17.5	495	0.0	43	495	0.0	–	495	0.0	4.5
p5.3.l	595	595	0.0	24.8	595	0.0	18	595	0.0	52.6	595	0.0	–	595	0.0	3.1
p5.3.n	755	755	0.0	30.4	755	0.0	17.1	755	0.0	54.8	755	0.0	–	755	0.0	3.7
p5.3.o	870	870	0.0	25.9	870	0.0	23.8	870	0.0	53.7	870	0.0	–	870	0.0	3.5
p5.3.q	1070	1070	0.0	25.6	1070	0.0	27.5	1070	0.0	63	1070	0.0	–	1070	0.0	7.8
p5.3.r	1125	1125	0.0	20.9	1125	0.0	21.8	1125	0.0	64.9	1125	0.0	–	1125	0.0	5.1
p5.3.s	1190	1190	0.0	36.2	1190	0.0	31.3	1190	0.0	63.8	1190	0.0	–	1190	0.0	0.5
p5.3.t	1260	1260	0.0	35	1260	0.0	28.9	1260	0.0	67.6	1260	0.0	–	1260	0.0	8.3
p5.3.u	1345	1345	0.0	32.5	1345	0.0	30	1345	0.0	69.2	1345	0.0	–	1345	0.0	5.8
p5.3.v	1425	1425	0.0	37.1	1425	0.0	31.5	1425	0.0	72.1	1425	0.0	–	1425	0.0	10.4
p5.3.w	1485	1470	1.01	46.8	1480	0.34	44.3	1485	0.0	73.7	1485	0.0	–	1485	0.0	20.1
p5.3.x	1555	1545	0.64	43.4	1550	0.32	43.9	1555	0.0	76.4	1555	0.0	–	1555	0.0	16.7
p5.3.y	1595	1590	0.31	51.9	1595	0.0	54.3	1595	0.0	79.8	1595	0.0	–	1595	0.0	25.2
p5.3.z	1635	1620	0.92	59.4	1635	0.0	33.7	1635	0.0	82.7	1635	0.0	–	1635	0.0	22.5
p5.4.m	555	555	0.0	15.2	555	0.0	8.6	555	0.0	36.5	555	0.0	–	555	0.0	5
p5.4.o	690	690	0.0	14	690	0.0	18.2	690	0.0	55.1	690	0.0	–	690	0.0	2.1
p5.4.p	765	765	0.0	16.9	765	0.0	15.1	765	0.0	68.3	765	0.0	–	765	0.0	3.4
p5.4.q	860	860	0.0	14.9	860	0.0	17.8	860	0.0	68.4	860	0.0	–	860	0.0	2.5
p5.4.r	960	960	0.0	32.8	960	0.0	33.7	960	0.0	66.9	960	0.0	–	960	0.0	6.1
p5.4.s	1030	1025	0.48	53.4	1030	0.0	48.8	1030	0.0	68.9	1030	0.0	–	1030	0.0	26.7
p5.4.t	1160	1160	0.0	19.3	1160	0.0	22.6	1160	0.0	66.1	1160	0.0	–	1160	0.0	5.3
p5.4.u	1300	1300	0.0	60.2	1300	0.0	60.0	1300	0.0	69.1	1300	0.0	–	1300	0.0	20.5
p5.4.v	1320	1320	0.0	65.2	1320	0.0	59.6	1320	0.0	74.8	1320	0.0	–	1320	0.0	25.5
p5.4.w	1390	1380	0.72	94.6	1385	0.36	98.1	1390	0.0	78.8	1390	0.0	–	1390	0.0	20.4
p5.4.x	1450	1435	1.03	135.8	1445	0.34	142.9	1450	0.0	82.2	1450	0.0	–	1450	0.0	21.2
p5.4.y	1520	1485	2.3	168.6	1520	0.0	161.3	1520	0.0	81.8	1520	0.0	–	1520	0.0	24
p5.4.z	1620	1620	0.0	182.1	1620	0.0	195.4	1620	0.0	85.9	1620	0.0	–	1620	0.0	33.2
AVG			0.23	43.3		0.04	40.5		0.07	66.4		0.0	11.6		0.0	10.1

Table 3

Detailed computational results of KSA on benchmark instances of set 6.

Inst		SHHS			SHHS2			LNS			HALNS			KSA		
Name	BKS	Best	Gap	CPU	Best	Gap	CPU	Best	Gap	CPU	Best	Gap	CPU	Best	Gap	CPU
p6.2.d	192	192	0.0	2.7	192	0.0	2.7	192	0.0	6.2	192	0.0	–	192	0.0	2.3
p6.2.j	948	948	0.0	9.8	948	0.0	10.2	948	0.0	52.1	948	0.0	–	948	0.0	6.2
p6.2.l	1116	1116	0.0	24.7	1116	0.0	21.3	1116	0.0	62.8	1116	0.0	–	1116	0.0	13.9
p6.2.m	1188	1188	0.0	49.2	1188	0.0	28.6	1188	0.0	65.9	1188	0.0	–	1188	0.0	10.5
p6.2.n	1260	1260	0.0	48.6	1260	0.0	51.2	1260	0.0	69.7	1260	0.0	–	1260	0.0	14.7
p6.3.g	282	282	0.0	7.3	282	0.0	4.4	282	0.0	7.7	282	0.0	–	282	0.0	4.2
p6.3.h	444	444	0.0	12.2	444	0.0	10.3	444	0.0	24.9	444	0.0	–	444	0.0	10.3
p6.3.i	642	642	0.0	12.0	642	0.0	8.3	642	0.0	39.8	642	0.0	–	642	0.0	8.3
p6.3.k	894	894	0.0	8.9	894	0.0	7.6	894	0.0	55.2	894	0.0	–	894	0.0	7.6
p6.3.l	1002	1002	0.0	12.9	1002	0.0	13.7	1002	0.0	55.2	1002	0.0	–	1002	0.0	11.3
p6.3.m	1080	1080	0.0	11.1	1080	0.0	10.3	1080	0.0	58.0	1080	0.0	–	1080	0.0	10.3
p6.3.n	1170	1170	0.0	16.8	1170	0.0	19.2	1170	0.0	62.4	1170	0.0	–	1170	0.0	13.2
p6.4.j	366	366	0.0	9.9	366	0.0	4.2	366	0.0	9.6	366	0.0	–	366	0.0	4.3
p6.4.k	528	528	0.0	13.7	528	0.0	14.8	528	0.0	26.3	528	0.0	–	528	0.0	9.5
p6.4.l	696	696	0.0	14.8	696	0.0	12.7	696	0.0	46.1	696	0.0	–	696	0.0	7.1
AVG			0.0	17		0.0	14.6		0.0	42.8		0.0	9.67		0.0	8.9

known solutions. However, it is not the case for the other tested

algorithms. LNS exhibits a relatively small ARPD to the optimal

solution of 0.0005% compared to 0.46% and 0.263% for HSSH2

and HSSH, respectively.

Table 4

Detailed computational results of KSA on benchmark instances of set 7.

Inst		SHHS			SHHS2			LNS			HALNS			KSA		
Name	BKS	Best	GAP	CPU	Best	GAP	CPU	Best	GAP	CPU	Best	GAP	CPU	Best	GAP	CPU
p7.2.d	190	190	0.00	3.6	190	0.00	3.1	190	0.0	4.4	190	0.0	–	190	0.0	2
p7.2.e	290	290	0.00	5.1	290	0.00	4.7	290	0.0	18.7	290	0.0	–	290	0.0	2
p7.2.f	387	384	0.78	20.3	387	0.00	31	387	0.0	42	387	0.0	–	387	0.0	5.8
p7.2.g	459	453	1.31	39.2	459	0.00	42	459	0.0	75	459	0.0	–	459	0.0	6.9
p7.2.h	521	520	0.19	61	521	0.00	68.3	521	0.0	92.8	521	0.0	–	521	0.0	3.3
p7.2.i	580	572	1.38	63.7	579	0.17	67.8	580	0.0	131.4	580	0.0	–	580	0.0	3.9
p7.2.j	646	637	1.39	77.8	641	0.77	82.9	646	0.0	137.7	646	0.0	–	646	0.0	33.9
p7.2.k	705	678	3.83	85.4	700	0.71	88.6	705	0.0	151.4	705	0.0	–	705	0.0	24.7
p7.2.l	767	742	3.26	84.9	761	0.78	92.1	767	0.0	155.1	767	0.0	–	767	0.0	27.9
p7.2.m	827	794	3.99	97.2	827	0.00	98.8	827	0.0	175.9	827	0.0	–	827	0.0	3.9
p7.2.n	888	859	3.27	102.5	878	1.13	105.7	888	0.0	181.4	888	0.0	–	888	0.0	57.8
p7.2.o	945	925	2.12	133.7	941	0.42	140	945	0.0	203.1	945	0.0	–	945	0.0	66.8
p7.2.p	1002	972	2.99	161.1	997	0.50	159.7	1002	0.0	208.7	1002	0.0	–	1002	0.0	70.8
p7.2.q	1044	1005	3.74	184.4	1043	0.10	189.3	1044	0.0	233.8	1044	0.0	–	1044	0.0	79.4
p7.2.r	1094	1052	3.84	198.8	1080	1.28	191.6	1094	0.0	247.2	1094	0.0	–	1094	0.0	69.1
p7.2.s	1136	1094	3.70	209.2	1125	0.97	213.8	1136	0.0	266.9	1136	0.0	–	1136	0.0	37.9
p7.2.t	1179	1128	4.33	205	1160	1.61	222.9	1179	0.0	283.5	1179	0.0	–	1179	0.0	59.4
p7.3.h	425	425	0.00	16.3	425	0.00	17.9	425	0.0	38.6	425	0.0	–	425	0.0	17.9
p7.3.i	487	487	0.00	17.4	487	0.00	20.2	487	0.0	59.9	487	0.0	–	487	0.0	2.3
p7.3.j	564	558	1.06	22.5	564	0.00	23.7	564	0.0	86.7	564	0.0	–	564	0.0	5.8
p7.3.k	633	632	0.16	29	633	0.00	27.2	633	0.0	121.7	633	0.0	–	633	0.0	9.3
p7.3.l	684	681	0.44	41.5	684	0.00	39.4	684	0.0	132.9	684	0.0	–	684	0.0	21.5
p7.3.m	762	762	0.00	39.5	762	0.00	41.1	762	0.0	160.8	762	0.0	–	762	0.0	23.2
p7.3.n	820	809	1.34	51.6	819	0.12	60	820	0.0	185	820	0.0	–	820	0.0	42.1
p7.3.o	874	856	2.06	63.2	874	0.00	68.8	874	0.0	202.1	874	0.0	–	874	0.0	50.9
p7.3.p	929	915	1.51	86.1	926	0.32	92.7	929	0.0	231.2	929	0.0	–	929	0.0	15.8
p7.3.q	987	972	1.52	97	987	0.00	103.4	987	0.0	235.1	987	0.0	–	987	0.0	17.5
p7.3.r	1026	1003	2.24	110.2	1019	0.68	118.6	1026	0.0	260.9	1026	0.0	–	1026	0.0	36.7
p7.3.s	1081	1056	2.31	148.7	1076	0.46	169.1	1081	0.0	261.9	1081	0.0	–	1081	0.0	3.1
p7.3.t	1120	1092	2.50	156.2	1114	0.54	140.1	1120	0.0	275.7	1120	0.0	–	1120	0.0	5.2
p7.4.g	217	217	0.00	7.9	217	0.00	8.4	217	0.0	3.7	217	0.0	–	217	0.0	3.1
p7.4.h	285	285	0.00	11	285	0.00	15.6	285	0.0	6.9	285	0.0	–	285	0.0	15.6
p7.4.i	366	366	0.00	19.1	366	0.00	19.8	366	0.0	16.4	366	0.0	–	366	0.0	4.8
p7.4.k	520	518	0.38	45.4	518	0.38	51.8	520	0.0	51.5	520	0.0	–	520	0.0	3
p7.4.l	590	576	2.37	69.6	590	0.00	73.5	590	0.0	73	590	0.0	–	590	0.0	61.5
p7.4.m	646	646	0.00	75.8	646	0.00	72.8	646	0.0	109.8	646	0.0	–	646	0.0	60.8
p7.4.n	730	726	0.55	102.5	726	0.55	99.3	730	0.0	141.2	730	0.0	–	730	0.0	2.3
p7.4.o	781	776	0.64	110.2	779	0.26	101	781	0.0	165.3	781	0.0	–	781	0.0	89
p7.4.p	846	834	1.42	165.5	846	0.00	109	846	0.0	183.9	846	0.0	–	846	0.0	97
p7.4.q	909	899	1.10	99.6	909	0.00	101.4	909	0.0	216.6	909	0.0	–	909	0.0	10.4
p7.4.r	970	952	1.86	109.8	970	0.00	100.7	970	0.0	228.2	970	0.0	–	970	0.0	28
p7.4.s	1022	1016	0.59	135.9	1022	0.00	128.2	1022	0.0	241.3	1022	0.0	–	1022	0.0	30.2
p7.4.t	1077	1063	1.30	172.6	1077	0.00	131.3	1077	0.0	277.6	1077	0.0	–	1077	0.0	11.3
AVG			1.52	86.91		0.27	86.91		0.0	152.95		0.0	30.89		0.0	28.46

Table 5

Number of best solutions found (#BKS), average relative percentage deviation (%ARPD) and average CPU time (sec) by set of instances.

	HSSH	HSSH2	LNS	HALNS	KSA
#BKS					
Set 4	9/53	25/53	53/53	53/53	53/53
Set 5	34/45	40/45	44/45	45/45	45/45
Set 6	15/15	15/15	15/15	15/15	15/15
Set 7	9/43	24/43	43/43	43/43	43/43
Total	67/156	104/156	155/156	156/156	156/156
%ARPD					
Set 4	0.565	0.0921	0	0	0
Set 5	0.067	0.0110	0.0021	0	0
Set 6	0	0	0	0	0
Set 7	0.42	0.08	0	0	0
AVG	0.263	0.046	0.0005	0	0
AVG CPU					
Set 4	107.3	109.6	218	32.24	31.49
Set 5	43.3	40.5	66.4	11.60	10.10
Set 6	17	14.6	42.8	9.67	8.9
Set 7	86.91	86.91	152.95	30.89	28.46
AVG	63.63	62.90	120.04	21.10	19.74

- The average CPU time is the last metric reported to evaluate and compare the results. We can observe that KSA is the best algorithm in relation to run time with an average CPU of 19.74 s.

The most competitive algorithm to KSA in terms of solution quality, is HALNS recorded 21.1 s. However, for HSSH, HSSH2 and LNS, the average CPU values are 62.9, 63.63 and 120.04 s.

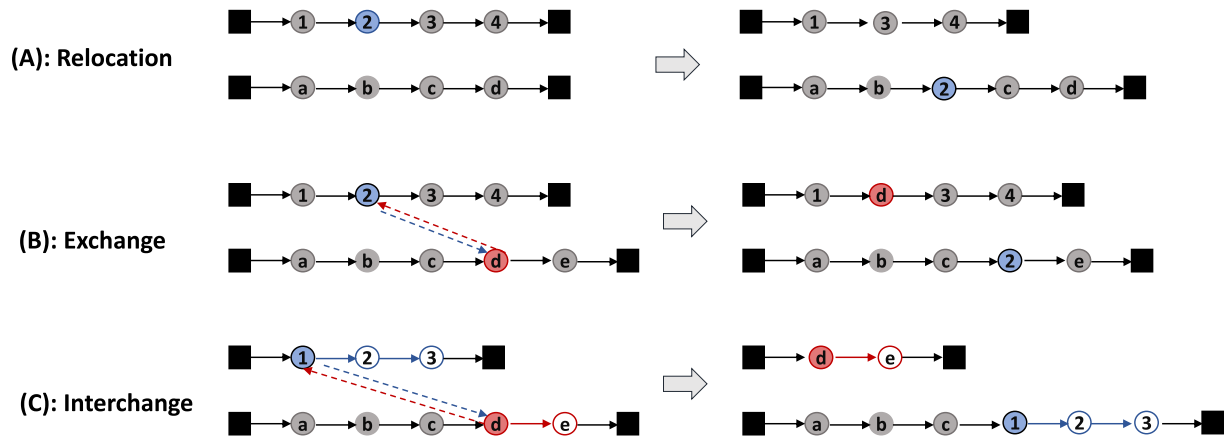


Fig. 3. Neighborhood operators.

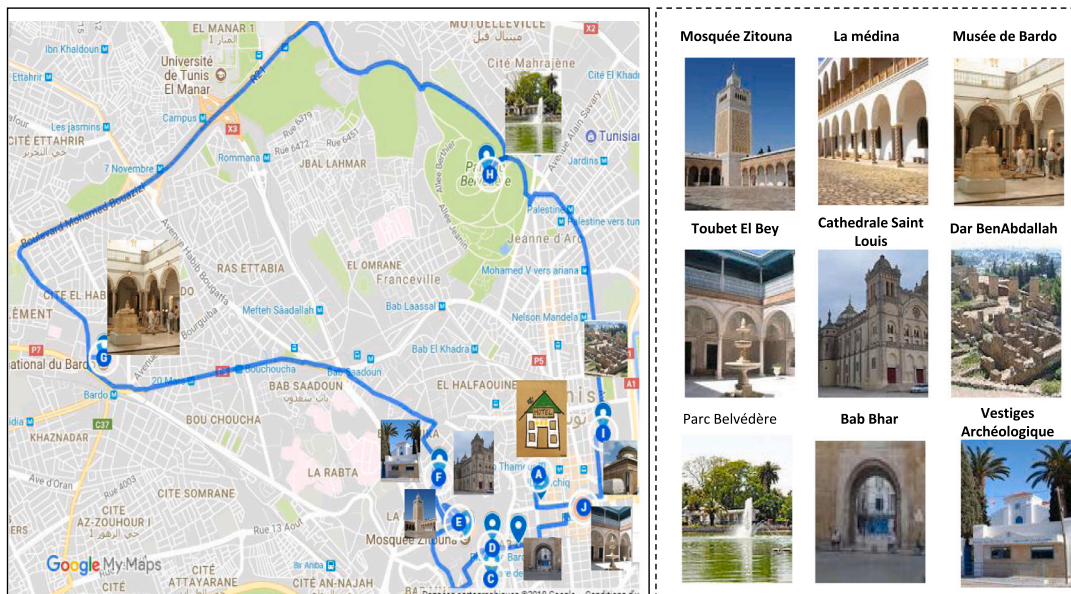


Fig. 4. Best selected POIs to visit by the tourist.

7. Real-world example

To demonstrate the various planning steps of our system, a short trip planning case in Tunis city at Tunisia. Tunis city is the sprawling capital of Tunisia. It attracts more than 2 millions tourists in 2016. In Tunis city, there are more than 57 POIs divided into museums, theaters, parks, monuments and historic sites. The target zone of the real case is composed of 9 places and one hotel which is the start point. Fig. 4 shows the chosen location of the studied example.

In this case, we test the KSA algorithm for 30 runs on real dataset of 57 POIs of Tunis divided into museums, theaters, parks, monuments and historic sights, etc, available in our database and extracted from TripAdvisor. A trip with 4 ratings, planned for day 1 starts from Africa Hotel at 9:00 AM and ends to Africa Hotel at 19:30 PM, where the tourist visits the museums and the historic sites of Tunis as respectively:

1. Bab Bhar
2. Mosquée Zitouna
3. Cathédrale Saint Louis
4. Vestiges Archéologique
5. Musée de Bardo
6. Parc Belvédère
7. Dar Ben Abdallah

8. La médina

9. Toubet El Bey

As shown, in Fig. 5, the plan of the trip is detailed in the table with start and end time of visiting in each POI. The map describes the short route that matches all the POIs to visit.

The developed TRS is a web-based personalized TRS which is developed using HTML5, CSS and JavaScript web technologies. The flowchart diagram of TRS interfaces is shown in Fig. 6.

8. Conclusions and future works

In this paper, a tourist recommender system (TRS) was developed to assist the tourists for solving the concern of planning their itinerary for their trips over a limited stay days. This issue can be viewed as a tourist trip design problem (TTDP) which is a well-known optimization problem having numerous extensions widely studied in the literature. The TTDP refers to a tour-planning problem for tourists concerned in visiting a set of points of interest (POIs) matching their preferences. The main objective of TTDP is to maximize the total collected scores of visited POIs. We proposed a cluster-first route-second metaheuristic (KSA) that combines k-means and simulated annealing methods. Our algorithm is integrated into the developed TRS to suggest efficiently

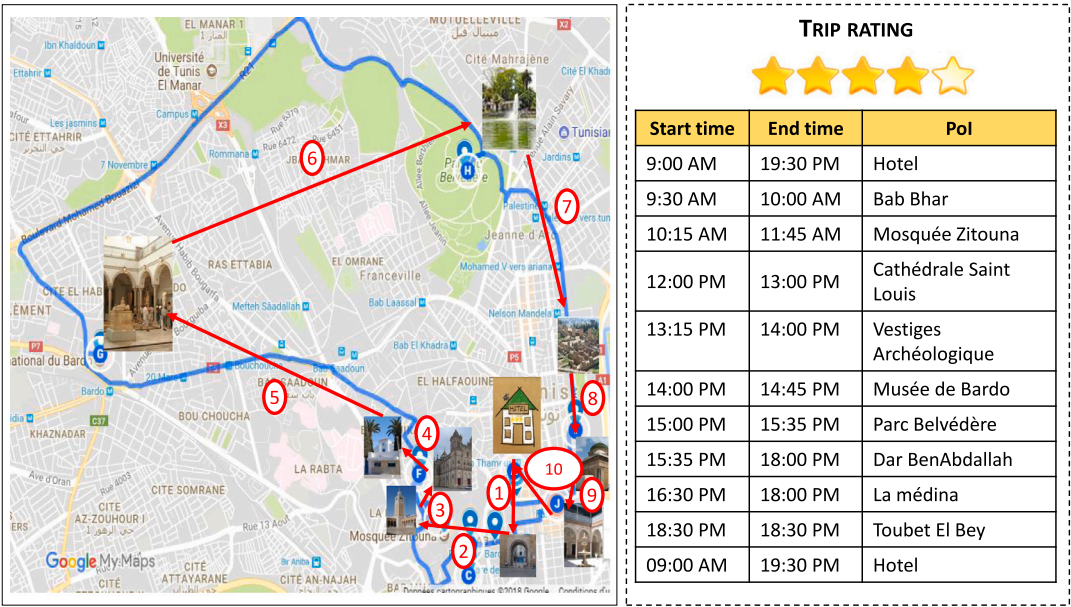


Fig. 5. A solution for 9 selected places for the first day.



Fig. 6. Flowchart diagram of StayPlan interfaces.

daily personalized route-planning for each user according to his own interests. We validated the effectiveness of our approach by performing numerical experiments. For this reason, we carried out a comparative study with four state-of-the-art algorithms using team orienteering problem benchmark. The results showed the outperforming and effectiveness of KSA algorithm in solving the TTDP in a faster time. Besides, a real-case example was illustrated in Tunis city to test the employment of StayPlan. The generated plan maximized the tourist satisfaction

by selecting attractions with higher ratings and taken into account each selected POI.

In order to further improve the performance of the system, more metaheuristic can be integrated into the optimizer engine to generate many solutions. Moreover, the framework will be more flexible if it tackles numerous variants of TTDP. Another suggestion is to run StayPlan on a real data set.

CRediT authorship contribution statement

Takwa Tlili: Conceptualization, Writing, Methodology, Experimental study. **Saoussen Krichen:** Reviewing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- Aghdasi, Hadi S., Saeedvand, Saeed, & Baltes, Jacky (2019). A multi-objective evolutionary hyper-heuristic algorithm for team-orienting problem with time windows regarding rescue applications. *The Knowledge Engineering Review*, 34.
- Bayliss, Christopher, Juan, Angel A., Currie, Christine S. M., & Panadero, Javier (2020). A learnheuristic approach for the team orienteering problem with aerial drone motion constraints. *Applied Soft Computing*, [ISSN: 1568-4946] 92, Article 106280. <http://dx.doi.org/10.1016/j.asoc.2020.106280>, URL <https://www.sciencedirect.com/science/article/pii/S1568494620302209>.
- Bianchessi, Nicola, Mansini, Renata, & Grazia Speranza, M. (2018). A branch-and-cut algorithm for the team orienteering problem. *International Transactions in Operational Research*, 25(2), 627–635.
- Bossek, Jakob, Grimme, Christian, Meisel, Stephan, Rudolph, Günter, & Trautmann, Heike (2019). Bi-objective orienteering: Towards a dynamic multi-objective evolutionary algorithm. In *International conference on evolutionary multi-criterion optimization* (pp. 516–528). Springer.
- Chao, I-Ming, Golden, Bruce L., & Wasil, Edward A. (1996). The team orienteering problem. *European Journal of Operational Research*, 88(3), 464–474.
- Chen, Jumei, & Shi, Jing (2019). A multi-compartment vehicle routing problem with time windows for urban distribution – a comparison study on particle swarm optimization algorithms. *Computers & Industrial Engineering*, [ISSN: 0360-8352] 133, 95–106. <http://dx.doi.org/10.1016/j.cie.2019.05.008>, URL <https://www.sciencedirect.com/science/article/pii/S0360835219302748>.
- Dibbelt, Julian, Konstantopoulos, Charalampos, Wagner, Dorothea, Gavalas, Damianos, Kontogiannis, Spyros, Zaroliagis, Christos, et al. (2017). Multimodal route and tour planning in urban environments. In *2017 IEEE symposium on computers and communications (ISCC)* (pp. 214–219). IEEE.
- Esmaeili, Leila, Mardani, Shahla, Hashemi Golpayegani, Seyyed Alireza, & Madar, Zeinab Zanganeh (2020). A novel tourism recommender system in the context of social commerce. *Expert Systems with Applications*, 149, Article 113301.
- Expósito, Aíram, Mancini, Simona, Brito, Julio, & Moreno, José A. (2019). A fuzzy grasp for the tourist trip design with clustered pois. *Expert Systems with Applications*, [ISSN: 0957-4174] 127, 210–227. <http://dx.doi.org/10.1016/j.eswa.2019.03.004>, URL <https://www.sciencedirect.com/science/article/pii/S0957417419301575>.
- Gavalas, Damianos, Kasapakis, Vlasios, Konstantopoulos, Charalampos, Pantziou, Grammati, Vathis, Nikolaos, & Zaroliagis, Christos (2014). A personalized multimodal tourist tour planner. In *Proceedings of the 13th international conference on mobile and ubiquitous multimedia* (pp. 73–80). ACM.
- Gavalas, Damianos, Konstantopoulos, Charalampos, Mastakas, Konstantinos, & Pantziou, Grammati (2014). A survey on algorithmic approaches for solving tourist trip design problems. *Journal of Heuristics*, 20(3), 291–328.
- Golden, Bruce L., Levy, Larry, & Vohra, Rakesh (1987). The orienteering problem. *Naval Research Logistics*, 34(3), 307–318.
- Gunawan, Aldy, Lau, Hoong Chuin, Vansteenwegen, Pieter, & Lu, Kun (2017). Well-tuned algorithms for the team orienteering problem with time windows. *Journal of the Operational Research Society*, 68(8), 861–876.
- Hammami, Farouk, Rekik, Monia, & Coelho, Leandro C. (2020). A hybrid adaptive large neighborhood search heuristic for the team orienteering problem. *Computers & Operations Research*, 123, Article 105034.
- Hu, Wanzhe, Fathi, Mahdi, & Pardalos, Panos M. (2018a). A multi-objective evolutionary algorithm based on decomposition and constraint programming for the multi-objective team orienteering problem with time windows. *Applied Soft Computing*, [ISSN: 1568-4946] 73, 383–393. <http://dx.doi.org/10.1016/j.asoc.2018.08.026>, URL <https://www.sciencedirect.com/science/article/pii/S1568494618304861>.
- Hu, Wanzhe, Fathi, Mahdi, & Pardalos, Panos M. (2018b). A multi-objective evolutionary algorithm based on decomposition and constraint programming for the multi-objective team orienteering problem with time windows. *Applied Soft Computing*, 73, 383–393.
- Jriji, Dorra, Krichen, Saoussen, & Madany, Fares (2020). A memetic algorithm for the tourist trip design with clustered points of interests. In *2020 international multi-conference on organization of knowledge and advanced technologies (OCTA)* (pp. 1–6). IEEE.
- Karabulut, Korhan, & Fatih Tasgetiren, M. (2020). An evolution strategy approach to the team orienteering problem with time windows. *Computers & Industrial Engineering*, [ISSN: 0360-8352] 139, Article 106109. <http://dx.doi.org/10.1016/j.cie.2019.106109>, URL <https://www.sciencedirect.com/science/article/pii/S0360835219305789>.
- Kim, Hyunjoon, Kim, Byung-In, & Noh, Dong jin (2020). The multi-profit orienteering problem. *Computers & Industrial Engineering*, [ISSN: 0360-8352] 149, Article 106808. <http://dx.doi.org/10.1016/j.cie.2020.106808>, URL <https://www.sciencedirect.com/science/article/pii/S0360835220305131>.
- Lin, Shih-Wei, & Vincent, F. Yu. (2015a). A simulated annealing heuristic for the multiconstraint team orienteering problem with multiple time windows. *Applied Soft Computing*, 37, 632–642.
- Lin, Shih-Wei, & Vincent, F. Yu. (2015b). A simulated annealing heuristic for the multiconstraint team orienteering problem with multiple time windows. *Applied Soft Computing*, [ISSN: 1568-4946] 37, 632–642. <http://dx.doi.org/10.1016/j.asoc.2015.08.058>, URL <https://www.sciencedirect.com/science/article/pii/S1568494615005712>.
- Logesh, R., Subramaniaswamy, V., & Vijayakumar, V. (2018). A personalised travel recommender system utilising social network profile and accurate gps data. *Electronic Government, an International Journal*, 14(1), 90–113.
- Ludewig, Malte, & Jannach, Dietmar (2018). Evaluation of session-based recommendation algorithms. *User Modeling and User-Adapted Interaction*, 28(4–5), 331–390.
- MacQueen, James, et al. (1967). Some methods for classification and analysis of multivariate observations. In *Proceedings of the fifth Berkeley symposium on mathematical statistics and probability*, Vol. 1, Oakland, CA, USA (pp. 281–297).
- Martin-Moreno, Rodrigo, & Vega-Rodríguez, Miguel A. (2018). Multi-objective artificial bee colony algorithm applied to the bi-objective orienteering problem. *Knowledge-Based Systems*, 154, 93–101.
- Metropolis, N., Rosenbluth, A., Rosenbluth, M., Teller, A., & Teller, E. (1953). Simulated annealing. *Journal of Chemical Physics*, 21, 1087–1092.
- Montaner, Miquel, López, Beatriz, & De La Rosa, Josep Lluís (2003). A taxonomy of recommender agents on the internet. *Artificial Intelligence Review*, 19(4), 285–330.
- Montejo-Ráez, Arturo, Perea-Ortega, José Manuel, García-Cumbreras, Miguel Ángel, & Martínez-Santiago, Fernando (2011). Otiüm: A web based planner for tourism and leisure. *Expert Systems with Applications*, 38(8), 10085–10093.
- Moreno, Antonio, Valls, Aida, Isern, David, Marin, Lucas, & Borràs, Joan (2013). Sigtur/e-destination: ontology-based personalized recommendation of tourism and leisure activities. *Engineering Applications of Artificial Intelligence*, 26(1), 633–651.
- Orlís, Christos, Bianchessi, Nicola, Roberti, Roberto, & Dullaert, Wout (2020). The team orienteering problem with overlaps: An application in cash logistics. *Transportation Science*, 54(2), 470–487.
- Pěnička, Robert, Faigl, Jan, Váňa, Petr, & Saska, Martin (2017). Dubins orienteering problem with neighborhoods. In *2017 international conference on unmanned aircraft systems (ICUAS)* (pp. 1555–1562). IEEE.
- Quadrana, Massimo, Cremonesi, Paolo, & Jannach, Dietmar (2018). Sequence-aware recommender systems. *ACM Computing Surveys*, 51(4), 1–36.
- Santini, Alberto (2019). An adaptive large neighbourhood search algorithm for the orienteering problem. *Expert Systems with Applications*, 123, 154–167.
- Schiaffino, Silvia, & Amandi, Analia (2009). Building an expert travel agent as a software agent. *Expert Systems with Applications*, 36(2), 1291–1299.
- Trachanatzi, Dimitra, Rigakis, Manousos, Marinaki, Magdalene, & Marinakis, Yannis (2020). An interactive preference-guided firefly algorithm for personalized tourist itineraries. *Expert Systems with Applications*, 159, Article 113563.
- Tsakirakis, Eleftherios, Marinaki, Magdalene, Marinakis, Yannis, & Matsatsinis, Nikolaos (2019). A similarity hybrid harmony search algorithm for the team orienteering problem. *Applied Soft Computing*, 80, 776–796.
- Umanets, Artem, Ferreira, Artur, & Leite, Nuno (2014). Guideme—a tourist guide with a recommender system and social interaction. *Procedia Technology*, 17, 407–414.
- Urrutia-Zambrana, Adolfo, Tirado, Gregorio, & Mateos, Alfonso (2021). Variable neighborhood search to solve the generalized orienteering problem. *International Transactions in Operational Research*, 28(1), 142–167.
- Vansteenwegen, Pieter, & Gunawan, Aldy (2019). Orienteering problems: Models and algorithms for vehicle routing problems with profits.
- Vansteenwegen, Pieter, Souffriau, Wouter, Berghe, Greet Vanden, & Van Oudheusden, Dirk (2011). The city trip planner: an expert system for tourists. *Expert Systems with Applications*, 38(6), 6540–6546.
- Vansteenwegen, Pieter, & Van Oudheusden, Dirk (2007). The mobile tourist guide: an opportunity. *OR Insight*, 20(3), 21–27.
- Verbeeck, Cédric, Vansteenwegen, Pieter, & Aghezzaf, E.-H. (2016). Solving the stochastic time-dependent orienteering problem with time windows. *European Journal of Operational Research*, 255(3), 699–718.
- Vincent, F. Yu., Perwira Redi, A. A. N., Jewpanya, Parida, & Gunawan, Aldy (2019). Selective discrete particle swarm optimization for the team orienteering problem with time windows and partial scores. *Computers & Industrial Engineering*, 138, Article 106084.
- Wörndl, Wolfgang, Hefe, Alexander, & Herzog, Daniel (2017). Recommending a sequence of interesting places for tourist trips. *Information Technology & Tourism*, 17(1), 31–54.
- Zhou, Yalan, Li, Chen, & Li, Yanyue (2018). A simulated annealing for multi-modal team orienteering problem with time windows. In *International conference on bio-inspired computing: Theories and applications* (pp. 23–30). Springer.